SRIC Real-LRM Pilot 0.1.0

Causal Audit Infrastructure for an External Multimode Coconut Model

Komorebi P.

September 3, 2026

Contents

SRIC Empirical Block R1 **2**
Real latent-reasoner causal-audit infrastructure . . . 2
Position in the SRIC sequence . . . 2
New object . . . 2
New empirical transaction . . . 3
Why this is the correct next block . . . 3
Claim boundary . . . 3

SRIC Real-LRM Pilot 0.1 Protocol **4**
1. Scientific question . . . 4
2. Why replay is the first gate . . . 4
3. Target runtime . . . 4
4. Protected outcome . . . 5
5. Task selection . . . 5
6. Procedure donors . . . 5
7. Semantic twins . . . 5
8. Interventions . . . 6
8.1 Zero trace . . . 6
8.2 Step permutation . . . 6
8.3 Norm-matched noise . . . 6
8.4 Procedure transplant . . . 6
8.5 Semantic-twin transplant . . . 6
8.6 Truncation . . . 6
9. Gate hierarchy . . . 6
`ADAPTER_INVALID` . . . 6
`LATENT_UNUSED` . . . 6
`LATENT_UNORDERED` . . . 7
`LATENT_MAGNITUDE_ONLY` . . . 7
`SURFACE_UNSTABLE` . . . 7
`LATENT_GENERIC` . . . 7
`PROCEDURE_SPECIFIC` . . . 7
10. Threshold discipline . . . 7
11. Statistical design . . . 7
12. Pre-audit seal . . . 7
13. Mandatory controls . . . 8
14. Threats to validity . . . 8
Formatting confound . . . 8
Cache confound . . . 8
Answer-only confound . . . 8
Surface-code confound . . . 8
Donor-quality confound . . . 8

Dataset leakage . . . 8
Semantic circularity . . . 9
15. Result interpretation . . . 9
16. Next empirical stage . . . 9

Multimode Coconut Adapter Contract **10**
Reviewed external interface . . . 10
Latent state being audited . . . 10
Required real adapter methods . . . 10
Replay obligation . . . 10
Teacher-forced sequence . . . 10
Batch and precision boundary . . . 11
Cache boundary . . . 11
Validation performed without the checkpoint . . . 11
Real compatibility gate . . . 11

Claim Boundary **12**
Established by this release . . . 12
Not established in this environment . . . 12
Maximum real Pilot 0.1 claim . . . 12
Synthetic fixture boundary . . . 12
Publication boundary . . . 12

Engineered Adapter Fixture Results **13**
Boundary . . . 13
Result . . . 13
Interpretation . . . 13

SRIC Empirical Block R1

Real latent-reasoner causal-audit infrastructure

Position in the SRIC sequence

Blocks 16 through 22 developed and validated causal semantics, chart growth, cross-architecture transport, obstruction-directed grammar growth, and correspondence-free intervention-orbit transport in controlled environments.

Empirical Block R1 changes the evidence source:

```
package-local constructed latent reasoner
                    ↓
third-party released latent-reasoning checkpoint
```

The block does not advance the cumulative theoretical version number because the external checkpoint has not yet been executed. It creates the sealed apparatus required for that execution.

New object

For each real task, the block records the exact sequence of hidden vectors that the external Coconut runtime reinjects into its latent-token positions:

`H(x) = (h_1(x), ..., h_K(x)).`

The primary causal object is not a descriptive hook. It is the state that actually enters the next recurrent computation.

New empirical transaction

```
freeze repository, checkpoint, dataset, config, and donors
        ↓
ordinary latent execution
        ↓
exact trace replay
        ↓
zero / order / noise interventions
        ↓
same-answer, different-proof transplantation
        ↓
semantic-twin transplantation
        ↓
first-failed status assignment
        ↓
receipt and preservation
```

Why this is the correct next block

A further synthetic positive block would add less evidential value than testing whether the framework survives an independently implemented model and independently produced weights. The immediate scientific question is no longer whether SRIC can define causal latent semantics. It is whether the audit can obtain a nontrivial and stable result on an external recurrent latent reasoner.

Claim boundary

The infrastructure is complete and internally validated. The external checkpoint result is pending execution on the target machine. The block must remain neutral between positive and negative checkpoint outcomes.

SRIC Real-LRM Pilot 0.1 Protocol

1. Scientific question

The pilot asks a deliberately narrow question:

> Does a released, third-party recurrent latent reasoner use a replayable latent trajectory that carries procedure-specific information beyond the final answer and remains stable under semantics-preserving surface transformations?

This is not equivalent to asking whether latent execution improves accuracy. It is also not equivalent to decoding a hidden state with a supervised probe. The target is a causal property of the actual recurrent trajectory.

For task `i`, let

```
H_i = (h_i1, ..., h_iK)
```

be the exact sequence of hidden states reinjected into the Coconut latent-token positions. Let `L_i(H)` be teacher-forced loss on the protected answer when trace `H` is supplied.

The primary estimands are:

```
necessity_i  = L_i(0)          - L_i(H_i)
order_i      = L_i(pi H_i)     - L_i(H_i)
structure_i  = L_i(eta H_i)    - L_i(H_i)
procedure_i  = L_i(H_j)        - L_i(H_i)
twin_i       = L_i(H_i_twin)   - L_i(H_i)
```

Here `j` is a different task with the same answer and approximately matched proof depth, selected to maximize proof-signature distance. `i_twin` is a surface-transformed version of the same logical problem.

A credible procedure-specific pattern is:

```
necessity > 0
order > 0
structure > 0
procedure > 0
twin approximately 0
```

2. Why replay is the first gate

The audit manipulates the recurrent state rather than a descriptive activation readout. If the captured trace cannot reproduce the ordinary computation, the intervention interface is invalid.

The replay gate compares both:

- answer loss under ordinary execution and captured-trace replay;
- answer-token logits under ordinary execution and captured-trace replay.

Failure terminates interpretation with `ADAPTER_INVALID`. No later effect is reported as evidence about the model's reasoning.

3. Target runtime

The primary target is the GPT-2 PrOntoQA multimode Coconut checkpoint released by the authors of the associated interpretability repository. The reviewed implementation supports three execution modes within one trained checkpoint:

- direct answer;
- verbalized chain of thought;
- latent hidden-state replacement.

Pilot 0.1 audits the latent mode and preserves the other two modes as later within-checkpoint controls.

The reviewed Coconut recurrence performs the following operation at each latent token:

1. run the causal language model through the token before the latent placeholder;
2. take the final-layer hidden state at that preceding position;
3. replace the latent placeholder's input embedding with that hidden state;
4. continue autoregressive computation with the updated embedding sequence.

The adapter mirrors this process and adds one capability: at each recurrence step, the naturally computed hidden state may be replaced by a supplied vector from a captured or intervened trace.

4. Protected outcome

The primary protected outcome is teacher-forced loss on the exact answer suffix:

```
### True
```

or

```
### False
```

including the external tokenizer's end-of-sequence token. Prompt, control tokens, attention mask, position IDs, answer suffix, and answer scoring positions are identical across intervention arms.

Greedy generated accuracy is a secondary behavioral metric. Teacher-forced answer loss is primary because it gives a continuous paired effect and avoids answer-extraction failures dominating the causal signal.

5. Task selection

The config declares a maximum task count. The selected subset is not the first rows in the dataset. Tasks are ranked by a deterministic SHA-256 function of the frozen seed and task ID, then the first `N` ranks are selected.

This prevents manual or accidental selection of unusually convenient examples. The complete dataset hash and selected task-ID digest are included in the pre-audit seal.

Tasks must be grouped by shared proof template or another dependence unit whenever such metadata is available. The current PrOntoQA loader defaults each original row to its own orbit but exposes `group_id` and `orbit_id` fields for a stronger grouped dataset export.

6. Procedure donors

For each recipient, donor selection follows a fixed rule:

1. require the same normalized final answer;
2. prefer the same proof depth;
3. exclude the same orbit;
4. maximize Jaccard distance between normalized proof signatures;
5. break ties by task ID.

The donor plan is generated before the model audit and stored in the seal through a canonical digest.

The proof steps are used for donor construction only. They are not used to train a semantic decoder, set intervention outcomes, or choose thresholds. A stronger later protocol will construct donor groups from formal proof graphs and will reveal graph labels only after the primary latent effects are fixed.

7. Semantic twins

The initial semantic twin conservatively reorders premise sentences while preserving:

- the query;
- final answer;
- proof steps;
- proof depth;
- proof signature;
- orbit and group identity.

The twin is not assumed valid merely because a string transformation ran. The package checks protected fields exactly. For the real study, additional twin families should be added only with an exact PrOntoQA verifier:

- entity renaming;
- predicate renaming;
- irrelevant-fact insertion;
- premise paraphrase;
- alternative but logically equivalent ordering.

The pilot's first admissible twin claim is therefore premise-order invariance, not unrestricted paraphrase invariance.

8. Interventions

8.1 Zero trace

Every reinjected vector is replaced by zero. This tests whether recurrent latent content contributes to answer scoring.

8.2 Step permutation

A deterministic derangement reorders the captured latent steps. It preserves the multiset of latent states while disrupting temporal order.

For one-step traces, this test is undefined. The primary configuration requires six latent positions.

8.3 Norm-matched noise

For every step, an independent Gaussian vector is normalized to the exact norm of the corresponding captured state. This controls for activation magnitude.

An optional covariance-matched arm samples from a covariance estimated only on the declared calibration split. It must not be estimated on final audit traces.

8.4 Procedure transplant

The full donor trajectory is substituted while retaining the recipient prompt and answer. The donor shares the final answer, reducing the answer-only confound.

8.5 Semantic-twin transplant

A trace from a premise-reordered semantic twin is substituted into the original task. The expected effect is near zero when the model's latent procedure is stable to the declared transformation.

8.6 Truncation

The package scores all prefixes from zero retained latent steps through the complete trace, zeroing subsequent states. The truncation curve is exploratory in Pilot 0.1 and becomes a primary stage-localization object only after a calibration study.

9. Gate hierarchy

The status hierarchy is deliberately sequential.

`ADAPTER_INVALID`

Exact trace replay exceeds the frozen tolerance.

`LATENT_UNUSED`

Replay succeeds, but zeroing the trace does not produce the predeclared effect and prevalence.

`LATENT_UNORDERED`

The latent trace contributes, but permuting its steps does not produce a material effect.

`LATENT_MAGNITUDE_ONLY`

Order matters, but norm-matched structured corruption does not outperform the declared floor.

`SURFACE_UNSTABLE`

The latent trace is useful and structured but changes materially under the declared semantics-preserving twin.

`LATENT_GENERIC`

The prior gates pass, but a same-answer, different-proof trace is nearly interchangeable with the task's own trace.

`PROCEDURE_SPECIFIC`

The same-answer procedure transplant has a positive mean effect and prevalence while semantic-twin transport remains within tolerance.

This last status does not identify the procedure's semantic content. It establishes only that the trajectory is task/procedure conditioned beyond the final answer under the declared donor and twin construction.

10. Threshold discipline

The distributed real config contains engineering defaults, not a completed scientific preregistration. Before the first final run:

1. use a disjoint adapter-smoke set to verify mechanics;
2. use a discovery/calibration split to estimate numerical noise and effect variance;
3. freeze thresholds and sample size;
4. create the pre-audit seal;
5. do not change the config after final execution begins.

A threshold change after final outcomes requires a new version, new seal, and new final support. The original result remains preserved.

11. Statistical design

All intervention effects are paired within tasks. The package reports:

- mean effect;
- positive-task fraction;
- 95% grouped bootstrap interval.

The bootstrap samples groups rather than assuming every transformed row is independent. The correct grouping level is the proof family, not the rendered text row.

Pilot 0.1 does not use the interval itself as the only gate. Mean magnitude and positive-task prevalence are separately required.

12. Pre-audit seal

The seal contains:

- complete audit configuration;
- external repository and exact commit;
- checkpoint repository and full checkpoint SHA-256;
- base-model reference;
- dataset SHA-256;
- selected task-ID digest;

- selected group-ID digest;
- donor-plan digest;
- maximum admissible claim;
- result schema;
- task count.

It contains no audit outcomes.

The real runner recomputes every available field and aborts on disagreement.

13. Mandatory controls

The first real result should be interpreted alongside:

- direct-answer mode accuracy;
- verbalized-CoT mode accuracy;
- zero-latent trace;
- random equal-norm trace;
- same-answer procedure donor;
- semantic-twin donor;
- different-answer donor as a weaker reference;
- wrong-depth donor;
- random same-answer donor;
- repeated execution determinism.

Pilot 0.1 implements the central latent interventions. Mode comparisons and expanded donor controls are staged for Pilot 0.2 after the replay contract is validated on the checkpoint.

14. Threats to validity

Formatting confound

Changing control tokens or answer positions can create an apparent latent effect. The adapter holds the complete sequence format fixed.

Cache confound

An intervention may disagree because cached key/value positions are inconsistent. Exact replay is the guard against this and the trace engine mirrors the pinned runtime's cache slicing.

Answer-only confound

A different-answer donor damages any trace that encodes the answer. The primary donor shares the answer.

Surface-code confound

A latent trace may encode prompt wording instead of logical procedure. The semantic-twin test checks one exact surface invariance.

Donor-quality confound

A donor may be too similar or differ in depth. The donor receipt records answer, depth, and proof-signature distance.

Dataset leakage

The checkpoint was trained by third parties and may have indirect exposure to task templates. The pilot is a mechanistic audit of the released checkpoint, not an untouched benchmark claim. Future audits require a new generator-foreign PrOntoQA support.

Semantic circularity

Gold proof steps used to construct donors cannot also independently validate an inferred semantic factor. Pilot 0.1 therefore stops at procedure specificity. Block 23 and Pilot 0.2 require a separate ontology-construction and post-seal witness split.

15. Result interpretation

A positive Pilot 0.1 result supports:

> Within the audited task support and execution format, the checkpoint uses a replayable, temporally structured latent trajectory whose replacement by a same-answer but different-proof trajectory damages answer scoring, while a declared semantic twin remains compatible.

It does not support:

- a unique English translation of each latent state;
- complete proof reconstruction;
- causal identification of particular proof variables;
- interpretability on unrestricted inputs;
- cross-model semantic equivalence.

16. Next empirical stage

Pilot 0.2 should begin only after the real Pilot 0.1 receipt exists. Its target is endogenous semantic-witness discovery:

1. construct anonymous intervention orbits without gold intermediate labels;
2. fit minimum-complexity predictive/interventional quotients;
3. seal the quotient and validation thresholds;
4. reveal formal proof graphs afterward;
5. test counterfactual latent editing;
6. transport any surviving factor to multimode CODI.

A `LATENT_UNUSED` or `LATENT_GENERIC` Pilot 0.1 result is not a reason to force Pilot 0.2. It is a valid checkpoint-specific obstruction certificate.

Multimode Coconut Adapter Contract

Reviewed external interface

The adapter is pinned to:

```
repository: connordilgren/are-lrms-easily-interpretable
commit: 32f413d8d55239d9bc54bb6b6ec37b0630891ed4
wrapper: models/multimode_coconut/multimode_coconut_model.py
runtime: src_coconut_multimode/coconut.py
checkpoint: connordilgren/gpt2-prontoqa-multimode-coconut / checkpoint_37
```

The package does not redistribute the external repository or model weights.

Latent state being audited

At latent position `k`, the external Coconut runtime:

1. computes the final-layer hidden state immediately before the latent placeholder;
2. reinjects that state as the placeholder's input embedding;
3. continues the causal computation.

Pilot 0.1 defines the latent trace as the ordered sequence of those **reinjected vectors**. It does not use an arbitrary transformer layer activation or a post-hoc mean pool.

Required real adapter methods

```
capture(task) -> TraceObservation
score_with_trace(task, trace) -> TraceObservation
model_fingerprint() -> Mapping[str, Any]
```

`capture` must perform ordinary latent recurrence and return the exact vectors inserted into the latent slots.

`score_with_trace` must preserve the recipient prompt, latent slot positions, answer suffix, attention mask, and position IDs while replacing only the reinjected vectors.

Replay obligation

For every task:

```
capture(task).answer_logits
```

must agree with:

```
score_with_trace(task, capture(task).trace).answer_logits
```

within the frozen tolerance. The package also compares protected answer loss.

Teacher-forced sequence

For GPT-2 multimode Coconut, the audited sequence is:

```
question + newline
<|bocot|>
<|latent|> repeated K times
<|eocot|>
### answer
EOS
```

Labels are `-100` through the latent prefix and equal the answer-token IDs afterward. This matches the external repository's answer formatting.

Batch and precision boundary

The external generation implementation supports batch size one. Pilot 0.1 preserves that boundary.

The initial replay should run in float32. Float16 or bfloat16 may be added only after a separate numerical-equivalence test and a new tolerance receipt.

Cache boundary

The custom trace engine mirrors the external runtime's legacy key/value-cache slicing. It accepts a newer cache object only when it can be converted to the legacy representation. An unsupported cache representation fails closed.

Validation performed without the checkpoint

The package contains a toy causal language model that exercises:

- multiple latent positions;
- natural trace capture;
- supplied trace replay;
- exact logit equality on replay;
- logit change under a modified trace.

This validates the intervention engine's control flow, not compatibility with every future Transformers release or the real checkpoint.

Real compatibility gate

The first real run must report:

- external commit equality;
- complete checkpoint SHA-256;
- successful model load;
- no nonfinite losses;
- exact trace shape `K x hidden_size`;
- replay gate pass on every audited task.

Any failure returns `ADAPTER_INVALID` or an execution error. It cannot be interpreted as evidence about the model's latent reasoning.

Claim Boundary

Established by this release

This release establishes that:

1. a replay-capable intervention interface has been implemented for the reviewed multimode Coconut recurrence;
2. the interface can capture and replace the exact hidden vectors reinjected at latent-token positions;
3. donor, semantic-twin, corruption, truncation, sealing, receipt, and grouped-bootstrap mechanics are executable;
4. an engineered procedure-specific fixture passes every declared gate;
5. a toy causal language model verifies exact replay and sensitivity to altered latent states;
6. the package can freeze a real checkpoint, dataset, donor plan, config, and maximum claim before execution.

Not established in this environment

The external GPT-2 PrOntoQA checkpoint was not downloaded or executed here. Therefore this release does not establish:

- checkpoint accuracy;
- latent necessity;
- temporal order sensitivity;
- proof-specific latent content;
- semantic-twin invariance;
- endogenous semantic discovery;
- causal semantic editing;
- cross-model transport.

Maximum real Pilot 0.1 claim

Even after a successful real run, the maximum claim is `PROCEDURE_SPECIFIC`.

This means that a same-answer, different-proof trace is not interchangeable with the task's own trace under the declared support and that a premise-order twin remains compatible. It does not identify a unique semantic interpretation of the trace.

Synthetic fixture boundary

The included `PROCEDURE_SPECIFIC` result is an engineered positive control. It validates the audit's ability to recognize a known procedure-specific trace. It is not evidence about a trained language model.

Publication boundary

Pilot 0.1 should be reported as:

- an infrastructure and preregistration artifact before the external run;
- a checkpoint-specific causal audit after the external run;
- not a general theorem about Coconut, latent reasoning, or Neuralese.

Engineered Adapter Fixture Results

Boundary

This is a deterministic positive control for the audit pipeline. It is not a trained language model and is not evidence about the external Coconut checkpoint.

Result

- Tasks: 48
- Status: `PROCEDURE_SPECIFIC`
- Maximum replay logit error: 0
- Maximum semantic-twin effect magnitude: 0
- Result receipt: `220324ce30387de4fb0216f315192551cffe51491e457ad15acd7533a1697fd4`

Effect	Mean	Grouped 95% interval	Positive fraction
Latent necessity	0.149230	[0.146630, 0.151482]	1.000
Step order	0.237738	[0.231035, 0.244419]	1.000
Norm-matched structure	0.346394	[0.334891, 0.357867]	1.000
Same-answer far-proof transplant	0.219653	[0.216013, 0.223558]	1.000
Semantic-twin transplant	0.000000	[0.000000, 0.000000]	0.000

Interpretation

The fixture was constructed so that its recurrent state is invariant to premise reordering, contains the final answer, and also contains step-indexed proof-procedure information. The audit correctly distinguishes a same-answer far-proof donor from a surface-only twin and returns `PROCEDURE_SPECIFIC`.

This validates the decision logic and intervention plumbing. It does not validate any empirical claim about a third-party checkpoint.